

PROGRAM NO – 46

AIM: To develop a Java program to create an interface having prototypes of functions area() and perimeter(). Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.

PROGRAM

```
import java.util.Scanner;

interface Shape
{
    double area();
    double perimeter();
}

class Circle implements Shape
{
    double radius;
    Circle(double r)
    {
        radius = r;
    }
    public double area()
    {
        return Math.PI * radius * radius;
    }
    public double perimeter()
```



```

    {
        return 2 * Math.PI * radius;
    }
}

class Rectangle implements Shape
{
    double length;
    double width;
    Rectangle(double l, double w)
    {
        length = l;
        width = w;
    }
    public double area()
    {
        return length * width;
    }
    public double perimeter()
    {
        return 2 * (length + width);
    }
}

public class InterfaceDemo3
{
    public static void main(String[] args)
    {
        Scanner s = new Scanner(System.in);
        while (true)
        {
            System.out.println("1. Circle");
            System.out.println("2. Rectangle");
            System.out.println("3. Exit");
            System.out.print("Choose an option(1-3): ");
            int choice = s.nextInt();
            switch (choice)

```

```

{
    case 1:
        System.out.print("Enter radius of circle: ");
        double r = s.nextDouble();
        Circle ob1 = new Circle(r);
        System.out.println("Area: " + ob1.area());
        System.out.println("Perimeter: " + ob1.perimeter());
        break;

    case 2:
        System.out.print("Enter length of rectangle: ");
        double l = s.nextDouble();
        System.out.print("Enter width of rectangle: ");
        double w = s.nextDouble();
        Rectangle ob2 = new Rectangle(l,w);
        System.out.println("Area of Rectangle: " + ob2.area());
        System.out.println("Perimeter of Rectangle: " + ob2.perimeter());
        break;

    case 3:
        System.out.println("Exiting...");
        s.close();
        System.exit(0);

    default:
        System.out.println("Invalid option. Please try again.");
}
}
}
}

```



Aim: To write a program to prepare bill with the given format using calculate method from interface

Order No. _____				
Date : _____				
Product Id	Name	Quantity	unit price	Total
101	A	2	25	50
102	B	1	100	100
Net. Amount				150

PROGRAM

```
import java.util.Scanner;

interface Calc
{
    void calculate();
}

class Bill implements Calc
{
    String pid,pname;
    int quantity;
    double price,total,amount=0;
    Scanner sc = new Scanner(System.in);
```

```

public void getdata()
{
    System.out.println("Enter product details ");
    System.out.println("=====");
    System.out.print("Product id: ");
    pid = sc.nextLine();
    System.out.print("Product name: ");
    pname = sc.nextLine();
    System.out.print("Quantity: ");
    quantity = sc.nextInt();
    System.out.print("Price: ");
    price = sc.nextDouble();
}

public void calculate()
{
    total = quantity * price;
}

public void display()
{
    System.out.println(pid+"\t "+pname+"\t "+quantity+"\t "+price+"\t "+total);
}
}

class Shop
{
    public static void main(String[] args)
    {
        int n,i,order_no;
        double namount=0;
        String date;
        Scanner s = new Scanner(System.in);
        System.out.println("Enter Bill details ");
        System.out.println("=====");
        System.out.print("Order No.");
        order_no=s.nextInt();
    }
}

```

```

    System.out.print("Date: ");
    date = s.next();
    System.out.print("No. of products: ");
    n = s.nextInt();
    Bill ob[] = new Bill[n];
    for(i=0;i<n;i++)
    ob[i] = new Bill();
    for(i=0;i<n;i++)
    {
        ob[i].getdata();
        ob[i].calculate();
    }
    System.out.println("Order No.: "+ order_no);
    System.out.println("Date: "+date);
    System.out.println("Productid Name Quantity Unitprice Total");
    System.out.println("===== ");
    for(i=0;i<n;i++)
    {
        ob[i].display();
        namount += ob[i].total; //namount=namount+ob[i].total;
    }
    System.out.println(" ");
    System.out.println("\t\t\tNet amount\t"+namount);
}
}

```



PROGRAM NO – C04-1

Aim: To write a Java program to create a package which contains a class to print a message. Import the package to main class & implement the method.

PROGRAM

Step 1: Create a package and a class

Create a folder(package) named "pkg1", then create a file named College.java inside the folder

// File: pkg1/College.java

```
package pkg1;

public class College
{
    public void msg()
    {
        System.out.println("Welcome to College!!");
    }
}
```

Step 2: Use the class from another file

Create a file named "PackageDemo.java" in the root directory (outside the "pkg1" folder):

// File: PackageDemo.java

```
import java.util.*;
```



```
import pkg1.College;
class PackageDemo
{
    public static void main(String args[])
    {
        College ob=new College();
        ob.msg();
    }
}
```

Compile and Run

```
javac pkg1/College.java
javac PackageDemo.java
java PackageDemo
```

OUTPUT

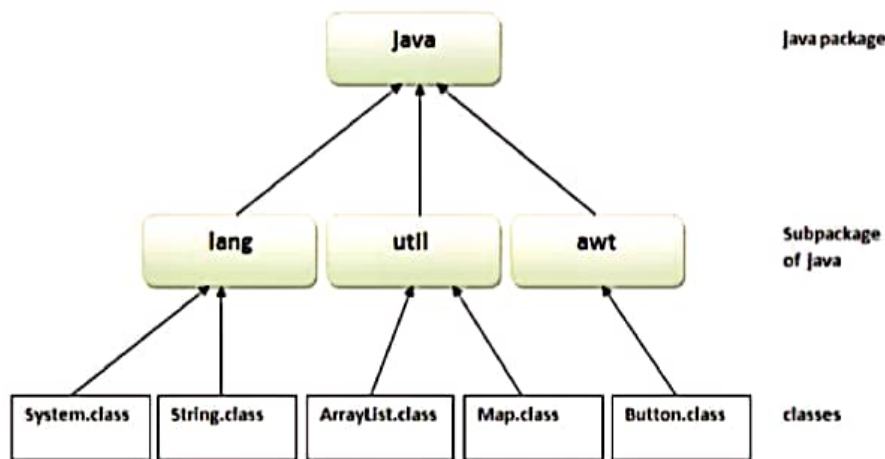
Welcome to College!!

Java Package

- A java package is a group of similar types of classes, interfaces and sub-packages.
- They compartmentalise class name space i.e. a packageA can have a class named ClassA while some other packageX can also have a class named ClassA.
- Packages are stored in a hierarchical manner and are explicitly imported into new class definitions.
- The package is both a naming and a visibility control mechanism.

Types of packages in Java

- 1) Built-in packages - java, lang, awt, javax, swing, net, io, util, sql



2) User-defined packages.

Creating a package

Include a package command as the 1st statement in the Java source file. Any classes declared within that file will belong to the specified package.

The general form of package statement is

```
package pkg;
```

pkg - name of user defined package.

Accessing packages

There are 3 ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

1. Using packagename.*

All classes and interfaces within a package can be accessed using packagename.* but subpackages i.e., packages within the package cannot be accessed.

Eg: import pkg1.*;

2. Using packagename.classname

Importing classes using package.classname makes only the declared class of this package accessible.

Eg: import pkg1.College;

3. Using fully qualified name

Using fully qualified name does not require import statement. But we need to use fully qualified name every time when we are accessing the class or interface. This way only declared class of this package will be accessible.

An Example

```
// pack/ A. java
```

```
package pack ;
```

```
public class A
```

```
{
```

```
    public void msg( )
```

```
    {
```

```
        System . out . println ( "Hello" );
```

```
    }
```

```
}
```

```
// B. java
```

```
class B
```

```
{
```

```
    public static void main ( String args [ ] )
```

```
    {
```

```
        pack . A obj = new pack . A ( ) ; // using fully qualified name
```

```
        obj . msg ( ) ;
```

```
    }
```

```
}
```

Advantages of Java Packages

1. Java package is used to categorize the classes and interfaces so that they can be easily maintained.
2. Java package provides access protection.
3. Java package removes naming collision.

PROGRAM NO – C04-1.1

Aim: To write a Java program to create a package named "pkg2" which contains a class "Hospital" to print a message. Import the package to main class & implement the method.

PROGRAM



```
javac pkg2/Hospital.java
java PackageDemo 2
```

OUTPUT

Muthoot Hospital!!

PROGRAM NO – C04-2

Aim: To write a program to create an arithmetic package that has classes & interfaces for basic arithmetic operation. Test the package by implementing all operations on two given number.

ALGORITHM

Algorithm:

Main Program

- 1 . Start
- 2 . Import required packages: Scanner and the custom operation classes (add, subtract, multiply, divide).
- 3 . Define a public class named "Arithmetic".
- 4 . Define the main method inside the arithmetic class.
- 5 . Create a Scanner object to read input from the user.
- 6 . Read first number from user and store it in variable "a".
- 7 . Read 2nd number from user and store it in variable "b".
- 8 . Create objects for the add, subtract, multiply, and divide classes.
- 9 . Call the "cal" method of each class using appropriate object, passing "a" and "b" as parameters
10. Stop

Interface -Calculate

- Define the interface "Calculate" in package with a method signature cal(int x, int y).

Classes-Add, Subtract, Multiply, Divide in package - pkgOperations

- Implement the method "cal" in classes that implement the calculate interface.
- Within the cal method, perform the desired calculation (addition, subtraction, multiplication, division) using the input parameters x and y and display the result.

PROGRAM

```
//pkgOperations/Calculate.java
```

```
package pkgOperations;

public interface Calculate
{
    void cal(int x, int y);
}
```

```
//pkgOperations/Add.java
```

```
package pkgOperations;

public class Add implements Calculate
{
    public void cal(int x, int y)
    {
        int sum = x+y;
        System.out.println("Sum of Numbers = "+sum);
    }
}
```

```
//pkgOperations/Subtract.java
```

```
package pkgOperations;

public class Subtract implements Calculate
{
    public void cal(int x, int y)
    {
        System.out.println("Difference of Numbers = "+(x-y));
    }
}
```



Aim: To write a program to create an arithmetic package that has classes & interfaces for basic arithmetic operation. Test the package by implementing all operations on two given number.

ALGORITHM

Algorithm:

Main Program

- 1 . Start
- 2 . Import required packages: Scanner and the custom operation classes (add, subtract, multiply, divide).
- 3 . Define a public class named "Arithmetic".
- 4 . Define the main method inside the arithmetic class.
- 5 . Create a Scanner object to read input from the user.
- 6 . Read first number from user and store it in variable "a".
- 7 . Read 2nd number from user and store it in variable "b".
- 8 . Create objects for the add, subtract, multiply, and divide classes.
- 9 . Call the "cal" method of each class using appropriate object, passing "a" and "b" as parameters
10. Stop

Interface -Calculate

- Define the interface "Calculate" in package with a method signature cal(int x, int y).

Classes-Add, Subtract, Multiply, Divide in package - pkgOperations

- Implement the method "cal" in classes that implement the calculate interface.
- Within the cal method, perform the desired calculation (addition, subtraction, multiplication, division) using the input parameters x and y and display the result.

PROGRAM

//pkgOperations/Calculate.java

```
package pkgOperations;

public interface Calculate
{
    void cal(int x, int y);
}
```

//pkgOperations/Add.java

```
package pkgOperations;

public class Add implements Calculate
{
    public void cal(int x, int y)
    {
        int sum = x+y;
        System.out.println("Sum of Numbers = "+sum);
    }
}
```

//pkgOperations/Subtract.java

```
package pkgOperations;

public class Subtract implements Calculate
{
    public void cal(int x, int y)
    {
        System.out.println("Difference of Numbers = "+(x-y));
    }
}
```

//pkgOperations/Multiply.java

```
package pkgOperations;

public class Multiply implements Calculate
{
    public void cal(int x, int y)
    {
        System.out.println("Multiplication of Numbers = "+(x*y));
    }
}
```



```

    {
        public void cal(int x, int y)
        {
            int sum = x+y;
            System.out.println("Sum of Numbers = "+sum);
        }
    }

```

//pkgOperations/Subtract.java

```

package pkgOperations;
public class Subtract implements Calculate
{

```

```

    public void cal(int x, int y)
    {
        System.out.println("Difference of Numbers = "+(x-y));
    }
}

```

//pkgOperations/Multiply.java

```

package pkgOperations;
public class Multiply implements Calculate
{
    public void cal(int x, int y)
    {
        System.out.println("Multiplication of Numbers = "+(x*y));
    }
}

```

//pkgOperations/Divide.java

```

package pkgOperations;
public class Divide implements Calculate
{
    public void cal(int x, int y)
    {
        System.out.println(x+"/"+y+" = "+(x/y));
    }
}

```

//Arithmetic.java

```

import java.util.Scanner;
import pkgOperations.Add;
import pkgOperations.Subtract;
import pkgOperations.Multiply;
import pkgOperations.Divide;
public class Arithmetic
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the first number: ");
        int a = sc.nextInt();
        System.out.println("Enter the second number: ");
        int b = sc.nextInt();
        Add objAdd = new Add();
        Subtract objSub = new Subtract();
        Multiply objMul = new Multiply();
        Divide objDiv = new Divide();
        objAdd.cal(a,b);
        objSub.cal(a,b);
        objMul.cal(a,b);
        objDiv.cal(a,b);
    }
}

```



```
import pkgOperations.Divide;

public class Arithmetic
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the first number: ");
        int a = sc.nextInt();
        System.out.println("Enter the second number: ");
        int b = sc.nextInt();
        Add obj1 = new Add();
        Subtract obj2 = new Subtract();
        Multiply obj3 = new Multiply();
        Divide obj4 = new Divide();
        obj1.cal(a,b);
        obj2.cal(a,b);
        obj3.cal(a,b);
        obj4.cal(a,b);
    }
}
```

Compiling Interface & Classes in package

javac pkgOperations/Calculate.java

javac pkgOperations/Add.java

javac pkgOperations/Subtract.java

javac pkgOperations/Multiply.java

javac pkgOperations/Divide.java

Compiling Main Program

javac Arithmetic.java

Run Main Program

java Arithmetic

PROGRAM NO – CO4-3

Aim: To write a Java program to create a "graphics" package that has classes and interfaces for figures circle, rectangle, square, triangle. Test the package by finding area of these figures.

PROGRAM

//create a folder graphics and create the following files in that

//graphics/Area_cal.java

```
package graphics;

public interface Area_cal
{
    void area();
}
```

//graphics/Circle.java

```
package graphics;
import java.util.Scanner;
public class Circle implements Area_cal
{
    double radius;
    public void area()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Input radius ");
        radius = sc.nextDouble();
        System.out.println("Area of circle: "+(3.14 *radius*radius));
    }
}
```

```
}  
  
//graphics/Rectangle.java
```

```
package graphics;  
import java.util.Scanner;  
public class Rectangle implements Area_cal  
{  
    int l,b;  
    public void area()  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Input length: ");  
        l = sc.nextInt();  
        System.out.println("Input breadth: ");  
        b = sc.nextInt();  
        System.out.println("Area of rectangle: "+(l*b));  
    }  
}
```

```
//graphics/Square.java
```

```
package graphics;  
import java.util.Scanner;  
public class Square implements Area_cal  
{  
    int a;  
    public void area()  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Input side length: ");  
        a = sc.nextInt();  
        System.out.println("Area of square: "+(a*a));  
    }  
}
```

```
}  
  
//graphics/Triangle.java
```

```
package graphics;  
import java.util.Scanner;  
public class Triangle implements Area_cal  
{  
    int b,h;  
    public void area()  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Input breadth: ");  
        b = sc.nextInt();  
        System.out.println("Input height: ");  
        h = sc.nextInt();  
        System.out.println("Area of triangle: "+(0.5*b*h));  
    }  
}
```

```
//Main Program-Root-GraphicsDemo.java
```

```
import graphics.Circle;  
import graphics.Rectangle;  
import graphics.Square;  
import graphics.Triangle;  
import java.util.Scanner;  
public class GraphicsDemo  
{  
    public static void main(String [] args)  
    {  
        Scanner sc = new Scanner(System.in);  
        int choice;  
        Circle obj1 = new Circle();
```

```
Rectangle obj2 = new Rectangle();
Square obj3 = new Square();
Triangle obj4 = new Triangle();
while(true)
{
    System.out.println("\n1)Circle\n2)Rectangle\n3)Square\n4)Triangle");
    System.out.print("Enter your choice(1-5):");
    choice = sc.nextInt();
    switch(choice)
    {
        case 1:
            obj1.area();
            break;
        case 2:
            obj2.area();
            break;
        case 3:
            obj3.area();
            break;
        case 4:
            obj4.area();
            break;
        case 5:
            System.out.println("Exiting!!");
            System.exit(0);
        default:
            break;
    }
}
}
```

}

Compiling

```
javac graphics/Area_cal.java  
javac graphics/Circle.java  
javac graphics/Rectangle.java  
javac graphics/Square.java  
javac graphics/Triangle.java  
javac GraphicsDemo.java
```

Run

```
java GraphicsDemo
```

PROGRAM NO – C04-4

Aim: To write a Java program to create a “shapes” package that has classes and interfaces for figures cube, cuboid, cylinder. Test the package by finding volume of these figures.

PROGRAM **Leave space as was needed for Pgm C04-3**



Compiling

```
javac shapes/Volume_cal.java
```

```
javac shapes/Cube.java
```

```
javac shapes/Cuboid.java
```

```
javac shapes/Cylinder.java
```

```
javac graphics/Triangle.java
```

```
javac ShapesDemo.java
```

Run

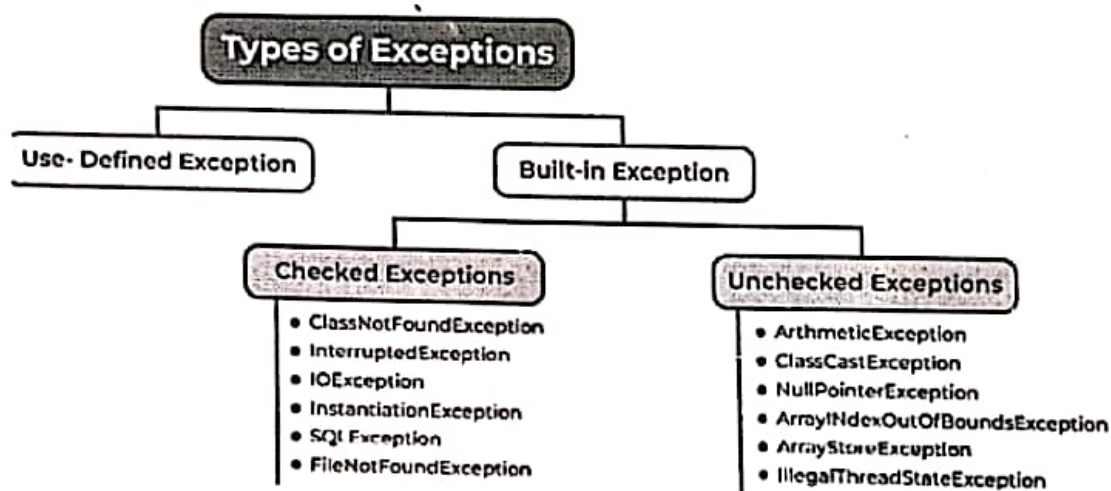
```
java ShapesDemo
```

Exception Handling in Java

- Exception is an abnormal condition that arises in a code at run-time.
- In Java, exception handling is one of the powerful mechanism to handle the runtime errors.

Exception

- In Java, an exception is an event that disrupts the normal flow of the program.
- It is an (event) object which is thrown at runtime.
- Thus, a Java exception is an object that describes the error condition that occurred in the code.
- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- At some point, these exceptions are caught and processed.



There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. 1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error: Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Keywords in exception handling

Java exception handling is managed via 5 keywords:

1. **try**: We may require to monitor certain program statements for exceptions. Such program statements are contained within a try block.
2. **catch**: If an exception occurs within the try block, it is thrown. This thrown exception is caught using catch and is handled in some manner.
3. **throw**: System-generated exceptions are automatically thrown by the Java run-time system. To manually throw an exception, we use keyword throw.
4. **throws**: Any exception that is thrown out from a method must be specified using throws clause.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Catching Exceptions (try/catch block)

- A method catches an exception using a combination of the try and catch keywords.
- A try/catch block is placed around the code that might generate an exception.
- Code within a try/catch block is referred to as protected code

Syntax for using try/catch Syntax

```
try
{
    // Protected code
}
catch (ExceptionName e1)
{
    // Catch block
}
```

- The code which is prone to exceptions is placed in the try block.
- When an exception occurs, that exception occurred is handled by catch block associated with it.
- Every try block should be immediately followed either by a catch block or finally block.

- A catch statement involves declaring the type of exception we are trying to catch.
- If an exception occurs in protected code, the catch block (or blocks) that follows the try is checked.
- If the type of exception that occurred is listed in a catch block, the exception is passed to the catch block much as an argument is passed into a method parameter.

1. Checked Exceptions

These are **checked at compile-time**. The programmer must handle them using `try-catch` or declare them using `throws`.

◆ Common Checked Exceptions:

Exception	Description
<code>IOException</code>	Signals I/O operation failure
<code>SQLException</code>	Error in database access
<code>FileNotFoundException</code>	File does not exist
<code>ClassNotFoundException</code>	Class not found during runtime
<code>InterruptedException</code>	Thread is interrupted while sleeping or waiting

2. Unchecked Exceptions (Runtime Exceptions)

These are **not checked at compile-time**, but occur during runtime.

◆ Common Unchecked Exceptions:

Exception	Description
<code>ArithmeticException</code>	Divide by zero or other illegal math ops
<code>NullPointerException</code>	Accessing method or field on a null object
<code>ArrayIndexOutOfBoundsException</code>	Accessing an array with invalid index
<code>NumberFormatException</code>	Invalid conversion from string to number
<code>IllegalArgumentException</code>	Method gets inappropriate argument
<code>IllegalStateException</code>	Object is in an incorrect state

Finally Block

- finally block follows a try block or a catch block.
- finally block of code always executes, irrespective of occurrence of an Exception.
- Using a finally block allows running any cleanup-type statements that needs to be executed, no matter what happens in the protected code.
- finally block appears at the end of the catch blocks.

```
try
{
    // Protected code
}
catch (ExceptionType1 e1)
{
    // Catch block
}
catch(ExceptionType2 e2)
{
    // Catch block
}
catch (ExceptionType3 e3)
{
    // Catch block
}
finally
{
```

- A catch clause cannot exist without a try statement.
- It is not mandatory to have finally clauses whenever a try/catch block is present.
- try block cannot be present without either catch clause or finally clause.
- No code can be present in between the try, catch and finally blocks.

throws/throw Keywords

- There are cases when our program requires to throw an exception explicitly.
- If a method does not handle a checked exception, the method must declare it using the throws keyword.
- The throws keyword appears at the end of a methods signature.

throw keyword

- We can throw an exception, either a newly instantiated one or an exception that we just caught, by using the throw keyword. While throws is used to postpone the handling of a checked exception, throw is used to invoke an exception explicitly. The following method declares that it throws a RemoteException :

Example:

```
import java.io.*;
public class className
{
    public void deposit (double amount ) throws RemoteException
    {
        // Method implementation
        throw new RemoteException( ) ;
    }
    // Remainder of class definition
}
```

throws keyword

A method can declare that it throws more than one exception, in which case the exceptions are declared as a list separated by commas.

Example, the following method declares that it throws a RemoteException and an InsufficientFundsException :-

```
import java.io.*;
public class className
{
    public void withdraw (double amount ) throws RemoteException,
    InsufficientFundsException
    {
        // Method implementation
    }
    // Remainder of class definition
}
```

Example: The following is an array declared with 2 elements. Then the code tries to access the 3rd element of the array which throws an exception.

```
// File Name : ExcepTest . java

public class ExcepTest
{
    static int a[] = {44,55} ;
    public static void main ( String args [ ] )
    {
        try
        {
            .....
            System.out.println("Access element three: " + a[3]) ;
        }
        catch ( ArrayIndexOutOfBoundsException e )
        {
            System.out.println("Exception thrown : " + e ) ;
        }
        finally
        {
            System.out.println("Access element two: " + a[1]) ;
        }
    }
}
```

OUTPUT

Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 3



Sample Program

Qn-Write a Java program to illustrate the case of "division by 0", give an appropriate error message and handle the exception.

```
public class DivException
{
    static int a=10,b=0;

    public static void main ( String args [] )
    {
        try
        {
            System.out.println("Quotient: " + (a/b)) ;
        }
        catch ( ArithmeticException e )
        {
            System.out.println("Exception thrown : " + e ) ;
        }
        finally
        {
            b++;
            System.out.println("Quotient: " + (a/b)) ;
        }
    }
}
```

Aim: To write a Java program to write a user defined exception class to authenticate the user name & password.

ALGORITHM

1. Start
- 2 . Define a class called "Authentication" with a "main" method.
- 2 . Set up username and password strings for authentication.
- 3 . Create a Scanner object to read input from the user.
- 4 . Read the username and password from the user.
- 5 . Use a try-catch block to handle potential exceptions.
- 6 . Check if the entered username and password match the predefined ones.
- 7 . If they match, grant access; otherwise, throw a user-defined exception "CredentialException" with the message "Invalid Credentials".
- 8 . Define the "CredentialException" class that extends the Exception class.
- 9 . Override the constructor to accept a message parameter and pass it to the superclass constructor.
10. Stop

PROGRAM

//CredentialException.java

```
public class CredentialException extends Exception
{
    public CredentialException(String s)
    {
        super(s);
    }
}
```

//Authentication.java

```
import java.util.Scanner;
public class Authentication
{
    public static void main(String[] args)
    {
        String username = "Ashish";
        String password = "@Ashish123";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the Username");
        String u1 = sc.nextLine();
        System.out.println("Enter the Password");
        String u2 = sc.nextLine();
        try
        {
```

```
        if ((u1.equals(username)) && (u2.equals(password)))
        {
            System.out.println("Access Granted");
        }
        else
        {
            throw new CredentialException("Invalid Credentials"); //User-defined Exception
            "CredentialException"
        }
    }
    catch(CredentialException e)
    {
        System.out.println(e.getMessage());
    }
}
}
```

COMPILE & RUN

javac CredentialException.java

javac Authentication.java

java Authentication



Aim: To write a Java program to find the average of N positive integers, raising a user defined exception for each negative input.

ALGORITHM

1. Start
2. Read the number of integers (`n`) from the user.
3. Initialize a variable `sum` to 0 to keep track of the sum of the entered integers.
4. Start a loop from 0 to `n-1` to read `n` integers from the user.
 - Read number (num) from the user.
 - Check if the input number is negative.
 - If the input is negative, throw a user-defined exception `InputException` with an appropriate error message
 - Otherwise, add it to the `sum`.
5. After the loop, calculate the average of the entered integers by dividing the sum by `n`. Ensure to cast one of the operands to `float` to perform floating-point division.
6. Print the calculated average.
7. Stop

PROGRAM

// InputException.java

```
//import java.util.Scanner;
class InputException extends Exception
{
    public InputException(String msg)
    {
        super(msg);
    }
}
```

//AverageException.java

```
public class AverageException
{
    public static void main(String[] args)
    {
        Scanner s = new Scanner(System.in);
        int sum = 0;
        System.out.print("Enter no of integers: ");
        int n = s.nextInt();
        // int arr[] = new int[n];
        System.out.println("Enter the integers ");
        try
        {
            for (int i = 0; i < n; i++)
            {
                int num = s.nextInt();
                if (num < 0)
                {
                    throw new InputException("Number is not positive !!!"); // throwing User-defined exception
                }
                sum+=num;
            }
            float avg = (float) sum / n; // Cast one operand to float to ensure floating point division
        }
        catch (InputException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

```
        System.out.println("Average is: " + avg);  
    }  
    catch (InputException e)  
    {  
        System.out.println(e);  
    }  
}
```

COMPILE & RUN

```
javac AverageException.java
```

```
java AverageException
```



Threads

- Threads are the smallest unit of dispatchable code.
- Threads are a way to make maximum use of the CPU, because idle time can be kept minimum.
- Java threads are objects like any other Java objects.
- Threads are instances of class `java.lang.Thread`, or instances of subclasses of this class.
- Java threads can also execute code.

Working

- A Java thread is like a virtual CPU that can execute our Java code - inside our Java application.
- When a Java application is started, its `main()` method is executed by the main thread - a special thread that is created by the Java VM to run our application.
- From inside the application we can create and start more threads which can execute parts of our application code in parallel with the main thread.
- We can execute parts of our single program concurrently. Each part can handle a different task at the same time making optimal use of the available resources specially when our computer has multiple CPU.
- Thus, threads are the smallest unit of dispatchable code.

Creating a thread

2 ways to create a thread:

1. **By extending Thread class**
2. By implementing Runnable interface.

1. Extending Thread class

Thread class provide constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Java Thread Example by extending Thread class

class Multi extends Thread

```
{
    public void run( )
    {
        System.out.println(" Thread i s running . . . " );
    }
    public static void main(String args[ ])
    {
        Multi t1=new Multi( );
        t1.start( );
    }
}
```

2. Runnable interface

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named run().

public void run(); is used to perform action for a thread.

Java Thread Example by implementing Runnable interface

class Multi3 implements Runnable

```
{  
    public void run()  
    {  
        System.out.println( " thread i s running . . . " );  
    }  
    public static void main(String args [ ] )  
    {  
        Multi3 m1=new Multi3();  
        Thread t1 =new Thread (m1 );  
        t1.start();  
    }  
}
```

Life cycle of a thread

start() method of Thread class is used to start a newly created thread.

It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.
 - o The thread is in **new state** if we create an instance of Thread class but before the invocation of start() method.
 - o The thread is in **runnable state** after invocation of **start()** method, but the thread scheduler has not selected it to be the running thread.
 - o The thread is in **running state** if the thread scheduler has selected it.
 - o **Non-Runnable (Blocked)** is the state when the thread is still alive, but is currently not eligible to run.
 - o A thread is in **terminated or dead state** when its run() method exits.

Commonly used methods of Thread class



There are a few commonly used methods when implementing threads.

1. `public void start()`: starts the execution of the thread. JVM calls the `run()` method on the thread.
2. `public void run()`: is used to perform action for a thread.
3. `public void sleep(long milliseconds)`: Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. `public void join()`: waits for a thread to die.

#####

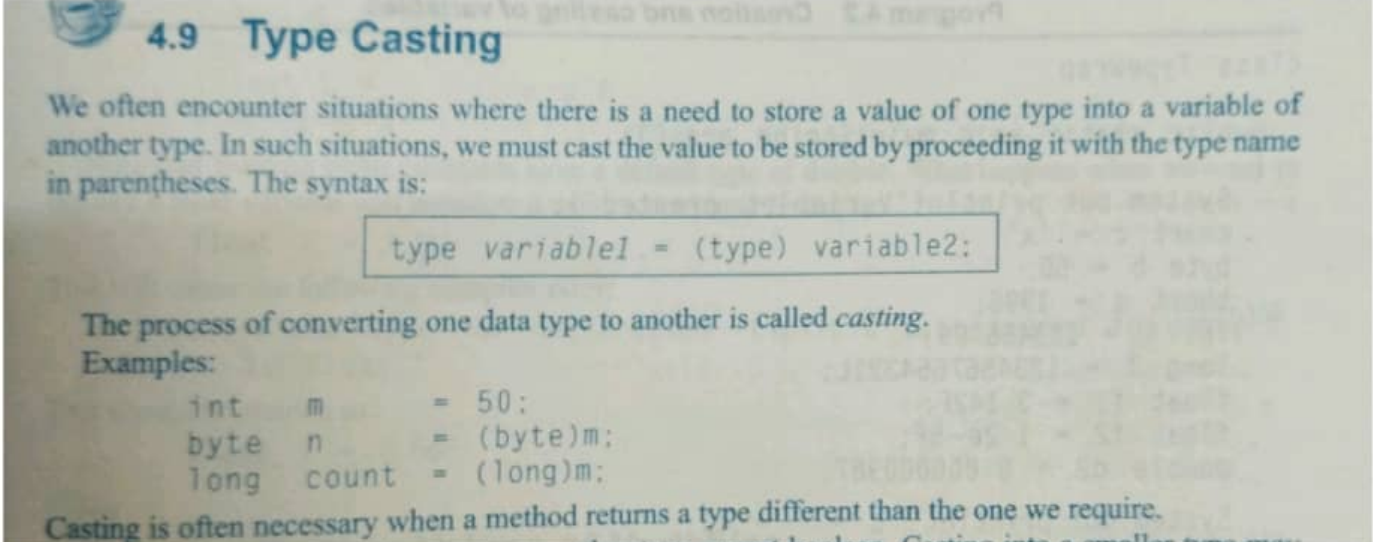
Static Methods in Java

Static methods belong to the **class** rather than to instances (objects) of the class. You can call a static method **without creating an object** of the class.

Key Characteristics:

- Declared using the **static** keyword.
- Can be called using the **class name** directly (e.g., `ClassName.methodName()`).
- **Cannot access instance variables** or instance methods directly – they can only access **other static members**.
- Commonly used for **utility or helper methods** that don't require any object state.

#####



4.9 Type Casting

We often encounter situations where there is a need to store a value of one type into a variable of another type. In such situations, we must cast the value to be stored by preceding it with the type name in parentheses. The syntax is:

```
type variable1 = (type) variable2;
```

The process of converting one data type to another is called *casting*.

Examples:

```
int    m    = 50;
byte   n    = (byte)m;
long   count = (long)m;
```

Casting is often necessary when a method returns a type different than the one we require.

PROGRAM NO – CO4-7

Aim: To write a Java program to define 2 classes; one for generating multiplication table of 5 and other for displaying first N prime numbers. Implement using threads. (Thread class).

ALGORITHM

1. Start
2. Define the Multiplication Thread:
 - a. The thread starts executing the run method.
 - b. In Run Method:
 - Print "Multiplication table of 5"
 - o Loop through numbers from 1 to 10.
 - o For each number i, print $i * 5$.
 - o Print "Exiting from Thread Multiplication...".
3. Define the PrimeNumbers Thread:
 - a. The thread starts executing the run method.
 - b. In Run Method:
 - Create a Scanner object to read input from the user.
 - Read the integer value n.
 - Call the prime_N method with n as the argument.
 - Close the Scanner object.
 - In prime_N Method:
 - o Print "All the Prime numbers within 1 and N are:".
 - o Use a flag variable for Prime checking
 - o Loop through numbers from 1 to N and inside it loop it from 2 to $x / 2$.
 - o If x is divisible by any of these numbers, set the flag to 0 and break the loop.
 - o If the flag is still 1, print x as it is a prime number.

4. Main Method in ThreadClass:

- a. Create an instance of the Multiplication class.
- b. Create an instance of the PrimeNumbers class.
- c. Start both threads by calling their start methods

5. Stop

PROGRAM

```
import java.util.Scanner;
class Multiplication extends Thread
{
    @Override
    public void run()
    {
        System.out.println("Multiplication table of 5:");
        for (int i = 1; i <= 10; i++)
        {
            System.out.println(i + " x 5 = " + i * 5);
        }
        System.out.println("Exiting from Thread Multiplication...");
    }
}
class PrimeNumbers extends Thread
{
    @Override
    public void run()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the value of n: ");
        int n = sc.nextInt();
        prime_N(n); // calling method prime_N(n) with argument "n"
        sc.close();
    }
    static void prime_N(int N)
    {
        int x, y, flag;
        System.out.println("All the Prime numbers within 1 and " + N + " are:");
        for (x = 2; x <= N; x++)
        {
            flag = 1;
            for (y = 2; y <= x / 2; ++y)
            {
                if (x % y == 0)
                {
                    flag = 0;
                    break;
                }
            }
            if (flag == 1)
            {
                System.out.print(x + "\t");
            }
        }
        System.out.println();
    }
}
public class ThreadClass
```

```
{  
    {  
        public static void main(String args[])  
        {  
            Multiplication a = new Multiplication();  
            PrimeNumbers b = new PrimeNumbers();  
            a.start();  
            b.start();  
        }  
    }  
}
```

Java AWT Basics

- Java AWT (Abstract Window Toolkit) is an API used to create Graphical User Interface (GUI) or Windows-based Java programs
- Java AWT components are platform-dependent, which means they are shown in accordance with the operating system's view.
- AWT is heavyweight, which means that its components consume resources from the underlying operating system (OS).
- The `java.awt` [package](#) contains AWT API classes such as *TextField*, *Label*, *TextArea*, *RadioButton*, *CheckBox*, *Choice*, *List*, and so on.

Event Handling in Java

- An **event** is a change in the state of an object triggered by some action such as Clicking a button, Moving the cursor, Pressing a key on the keyboard, Scrolling a page, etc.
- In Java, the `java.awt.event` package provides various event classes to handle these actions.

Event Handling Mechanism

Event handling is a mechanism that allows programs to control events and define what should happen when an event occurs. Java uses the **Delegation Event Model** to handle events. This model consists of two main components:

- **Source:** Events are generated from the source. There are various sources like buttons,



checkboxes, list, menu-item, choice, scrollbar, text components, windows, etc., to generate events.

- **Listeners:** Listeners are used for handling the events generated from the source. Each of these listeners represents interfaces that are responsible for handling events.

Registering the Source With Listener

To handle events, the source must be registered with a listener. Java provides specific methods for registering listeners based on the type of event.

Syntax:

`addTypeListener`
^

For example,

- `addKeyListener()` for `KeyEvent`
- `addActionListener()` for `ActionEvent`

Event Classes and Listener Interfaces

Java provides a variety of event classes and corresponding listener interfaces. Below table demonstrates the most commonly used event classes and their associated listener interfaces:

Event Class	Listener Interface	Description
ActionEvent	ActionListener	An event that indicates that a component-defined action occurred like a button click or selecting an item from the menu-item list.
AdjustmentEvent	AdjustmentListener	The adjustment event is emitted by an Adjustable object like Scrollbar.
ComponentEvent	ComponentListener	An event that indicates that a component moved, the size changed or changed its visibility.
ContainerEvent	ContainerListener	When a component is added to a container (or) removed from it, then this event is generated by a container object.
FocusEvent	FocusListener	These are focus-related events, which include focus, focusin, focusout, and blur.
ItemEvent	ItemListener	An event that indicates whether an item was selected or not.
KeyEvent	KeyListener	An event that occurs due to a sequence of keypresses on the keyboard.
MouseEvent	MouseListener & MouseMotionListener	The events that occur due to the user interaction with the mouse (Pointing Device).
MouseWheelEvent	MouseWheelListener	An event that specifies that the mouse wheel was rotated in a component.
TextEvent	TextListener	An event that occurs when an object's text changes.

Methods in Listener Interfaces

Each listener interface contains specific methods that must be implemented to handle events. Below table demonstrates the key methods for each interface:

Listener Interface	Methods
ActionListener	actionPerformed()
AdjustmentListener	adjustmentValueChanged()
ComponentListener	componentResized() componentShown() componentMoved() componentHidden()
ContainerListener	componentAdded() componentRemoved()
FocusListener	focusGained() focusLost()
ItemListener	itemStateChanged()
KeyListener	keyTyped() keyPressed() keyReleased()
MouseListener	mousePressed() mouseClicked() mouseEntered() mouseExited() mouseReleased()

*

Frame

Frame is a class in java.awt used to create GUI windows. It is a **container** that can hold components like Button, Label, TextField, etc.

Key Features:

- It provides a **window** for your application.
- It can be shown, hidden, resized, titled, and closed.
- You can add components to it using layout managers.

Panel

Panel is a container class used to organize and hold components (like buttons, labels, etc.). It doesn't have a border or title and is typically used to group components within a frame.



PROGRAM NO – C05-1

Aim: To write a Java program to find maximum of three numbers using AWT.

ALGORITHM

1. Start
2. Import required AWT classes.
3. Define a class named Max3.
4. Inside the class:
 - a. Declare necessary variables.
 - b. Set layout to FlowLayout.
 - c. Define a constructor to initialize the GUI components.
 - d. Create Labels for l1, l2, l3 and lMax.
 - e. Create TextFields for t1, t2, t3, and tResult.
 - f. Create a Button for bFindMax.
 - g. Add components to the frame.
 - h. Add ActionListener to the bFindMax button.
 - i. Set frame properties: title, size, and visibility
 - j. Inside actionPerformed() method:
 - Get input values from t1, t2, and t3 TextFields.
 - Calculate the maximum of the 3 numbers using Math.max.
 - Set the tResult TextField with the maximum value.
 - k. Define main method to create an instance of Max3 class.

5. Stop

PROGRAM

```
import java.awt.*;
import java.awt.event.*;

public class Max3 extends Frame implements ActionListener // Max3 extends Frame → it becomes a window.
{
    // Implements ActionListener → allows it to respond to button clicks.
    TextField t1, t2, t3, tResult;
    Label l1, l2, l3, lMax;
    Button bFindMax;
    public Max3() // Max3() constructor
    {
        setLayout(new FlowLayout()); // Arranges components left to right, top to bottom
```



```

l1=new Label("Number 1");
l2=new Label("Number 2");
l3=new Label("Number 3");
lMax=new Label("Maximum");

t1 = new TextField(10);
t2 = new TextField(10);
t3 = new TextField(10);
tResult = new TextField(10);

bFindMax = new Button("Find Max");
//result.setEditable(false);

add(l1);                                // Adds GUI components
add(t1);

add(l2);
add(t2);

add(l3);
add(t3);

add(bFindMax);

add(lMax);
add(tResult);

bFindMax.addActionListener(this);      // Add event listener – "ActionListener"

setTitle("Find Maximum of Three Numbers"); // Frame settings-Sets window title, size, and makes it
visible.
setSize(250, 200);
setVisible(true);

addWindowListener(new WindowAdapter()  // Handle window closing
{
    public void windowClosing(WindowEvent we)
    {
        System.exit(0);
    }
}

```

```

    });

}

public void actionPerformed(ActionEvent ae) //Event Handling-This method is triggered when the button is
clicked.
{
    int n1,n2,n3,max;

    n1 = Integer.parseInt(t1.getText()); // Read values from t1, t2, and t3 using getText()

    n2 = Integer.parseInt(t2.getText()); // Convert them to integers using Integer.parseInt()

    n3 = Integer.parseInt(t3.getText());
    max = Math.max(n1, Math.max(n2, n3)); // finds the maximum of 3 numbers using just one line of code with nested
// Math.max() calls.

    tResult.setText(String.valueOf(max)); // Display result in tResult
}

public static void main(String[] args)
{

    new Max3(); // Starts the program by creating an instance of the Max3 class.

    //Constructor runs, which sets up the GUI.

    //U created the object, but u didn't assign it to any reference variable. That's
perfectly
    //valid in Java if you don't need to use the object again later. If you just want to trigger
the
    //constructor (i.e., set up the window) and don't need to access the object again, then it's
OK not to
    //assign it to a variable.

}

}

```